49. RAG Service in Go: Chunk → Embed → Retrieve → Generate

49. RAG Service in Go: Chunk → Embed → Retrieve → Generate

Phase: Phase 7 — AI engineering in Go
Estimated time: 2–3 days
Slug: `49-rag-service`

Learning objectives

- Chunk text with overlap
- Orchestrate embed+retrieve+generate
- Cite sources in answers
- Evaluate retrieval quality

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

RAG systems chunk documents, embed, retrieve, then generate. Go is excellent for the retrieval service path.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Chunk with overlap to preserve context.
- Embed chunks offline or online.
- Retrieve top-k by similarity.
- Prompt = instructions + retrieved context + question.
- Cite sources for trustworthiness.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-rag && cd /tmp/gw-rag
go mod init example.com/scratch
```

Then create `main.go`:

```
package main

import "fmt"

func Chunk(s string, size int) []string {
    var out []string
    for i := 0; i < len(s); i += size {
        j := i + size
        if j > len(s) {
            j = len(s)
        }
        out = append(out, s[i:j])
    }
    return out
}

func main() { fmt.Println(Chunk("abcdefghij", 4)) }
```

Run it:

```
go run .
```

Expected output:

```
[abcd efgh ij]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Chunk with overlap (e.g., size 4, overlap 1).
2. Retrieve top-1 from a toy map of embeddings.
3. Build a prompt string from docs + question.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

RAG pipeline

1. **Chunk** documents (size + overlap)
2. **Embed** chunks → vectors
3. On query: embed query → **retrieve** top-K chunks
4. **Generate** answer with LLM using retrieved context
5. Return answer + citations

Go orchestration

Go shines as the reliable orchestrator: timeouts, concurrency, observability, auth.

```
chunks := Chunk(doc, 500, 50)
// embed + store
hits := store.TopK(ctx, queryVec, 5)
answer, err := llm.Generate(ctx, prompt(hits, question))
```

Keep prompts and models versioned. Measure hit rate and groundedness.

Failure modes to watch while coding

- Huge chunks blowing context windows
- No citations
- Retrieving without metadata filters

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- LangChain glue vs explicit Go pipelines — explicit often clearer for prod.
- Same RAG diagram.
- Evaluate retrieval quality separately from generation.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Bad chunking dominates RAG quality more than model choice for many corpora.

Practice checklist

Before the exercise, confirm you can:

1. Explain “RAG Service in Go: Chunk → Embed → Retrieve → Generate” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Huge chunks blowing context windows
- No citations
- Retrieving without metadata filters

Exercises

Work in `exercises/49-rag-service/`. Read that folder's `README.md` before you code.

What you will build

Implement chunk/retrieve helpers tests require.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/49-rag-service/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why overlap in chunking?
2. What are citations for?
3. Where does Go sit in RAG?

Further reading

- https://www.anthropic.com/engineering